Exam-Flow: An Agentic Multimodal Framework for Automated Academic Integrity Verification

Motasem Aldiab
School of Computing and Academic Studies
British Columbia Institute of Technology
Vancouver, Canada
maldiab@bcit.ca

M. Sajid Ali
Faculty of Information Technology
Preston University
Islamabad, Pakistan
ORCID: 0000-0001-6728-9118

***Abstract*—Remote programming assessments provide several kinds of evidence: recorded student behaviour, code development on screen, and the submitted program. Exam-Flow combines these sources through four software agents: a Visual Proctor, a Code Reviewer, an Integrity Auditor, and a Final Grader. This paper presents the framework as a proof of concept through four demonstration cases recorded by one developer. The saved case summary records PASS for an honest case and FAIL for two anomaly cases. The fourth case has an INVESTIGATION audit status followed by a grading penalty. These observations show how the system generates reports and why audit status and grading outcome must be distinguished. They do not establish detection accuracy, a false-alarm rate, or statistical significance. We describe the available implementation, explain limitations in event timing and outcome scoring, and distinguish new analysis from retrieval of stored reports. The contribution is a prototype for examining related evidence. Broader validation and safeguards for reviewing decisions are still needed.**

***Index Terms*—academic integrity, multimodal proctoring, agentic systems, proof of concept, large language models**

I. Introduction

Remote programming assessments require instructors to evaluate both a submitted solution and how it was produced. Research on online assessment and proctoring has identified concerns about misconduct, privacy, and the interpretation of student behaviour [1]–[3].

Video recordings and code comparison provide different kinds of evidence. A recording may show a student looking away from the screen, but it cannot establish the reason for that action. Code comparison can identify similarities between submissions, but the final program alone does not show when its contents were entered [5], [6]. Reviewing the coding process alongside the submitted program may help instructors identify discrepancies that need further explanation.

We use *forensic triangulation* to mean examining behavioural observations, screen activity, and submitted code together. A code insertion or gaze shift alone is insufficient evidence of misconduct. Legitimate pasting, refactoring, and looking at a keyboard can produce similar signals. The value of combining these sources must therefore be tested with both suspicious and ordinary behaviour.

This paper introduces Exam-Flow as a proof of concept. Its contributions are:

1) A four-agent architecture that combines behavioural reports, code review, integrity assessment, and grading.
2) A method for combining timestamped observations and code discrepancies in a report that an instructor can review.
3) A description of four selected cases and the limitations revealed by inconsistent outcome labels and event timing.

The study does not establish that this architecture improves accuracy over individual detectors. It describes the prototype workflow and preliminary observations from the saved case records.

II. Related Work

A. Visual Behavioral Proctoring and Its Ethical Limits

Automated proctoring can use face detection, head position, and other video observations to identify events for review [2], [9]. Interpreting an event as misconduct requires context. Ethical studies raise concerns about privacy, student autonomy, and the consequences of incorrect allegations [3], [4]. These concerns support the need for reports that can be checked and for a clear distinction between observations and disciplinary decisions.

B. Static Code Analysis and the Generative-AI Challenge

Tools such as JPlag compare similarities between source files [5]. Reviews also describe methods that compare program structure and behaviour to detect code plagiarism [6], [7]. Similarity alone does not explain how a student developed a program. Generative AI provides another reason to examine the development process alongside the final program. This paper does not evaluate AI authorship detection or establish that any particular program was generated by AI.

C. Multimodal Fusion and Agentic Orchestration

Exam-Flow assigns different review tasks to separate agents and passes their reports to an auditor. This makes each agent's role clear, but the agents are not independent witnesses. They can share inputs, models, and errors, so agreement between them cannot be treated as independent confirmation. Retrieval-augmented generation supplies reference material to

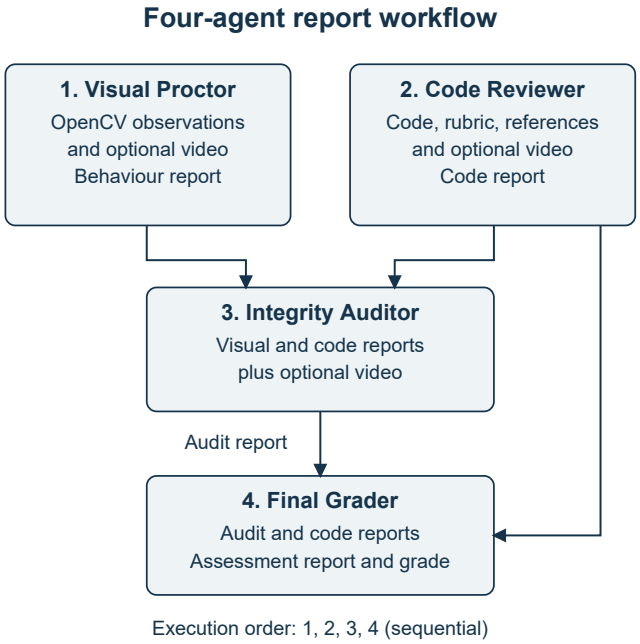


Fig. 1. Four-agent workflow in the inspected implementation. Arrows show report inputs; calls run sequentially in numbered order. The Final Grader uses the audit and code reports.

a language model [8]. Its effect on assessment quality in Exam-Flow still requires measurement.

The Community of Inquiry framework describes cognitive, social, and teaching presence in learning environments [11]. It gives a reason to consider the learning context, but this study does not measure those concepts. Monitoring a face does not measure engagement, and an automated integrity report should not replace an instructor's judgment.

D. Summary and Research Gap

The problem is how to present related behavioural and code evidence in one assessment workflow. Exam-Flow explores this through a working prototype and selected cases. Measuring its value requires a common dataset, documented comparison methods, and tests of both misconduct scenarios and legitimate activities that resemble them.

III. The Exam-Flow Framework

The workflow includes retrieving reference material, preparing recorded evidence, analysing it through the agents, and combining their reports. Figure 1 shows how the evidence sources are connected. In the code reviewed for this study, the agents run one after another, with delays between calls. The diagram shows information flow; the agents do not run concurrently in this implementation. The prototype analyses uploaded recordings and source files. The case records do not establish live detection performance.

A. Agentic Roles

The **Visual Proctor (VP)** receives OpenCV observations and, when available, a video file. Its prompt asks for an anomaly log and an explanation of face absence and gaze shifts. The project notes report a sampling interval of 0.2 seconds, or 5 frames per second. This setting alone does not establish the accuracy of the event times reported by the model.

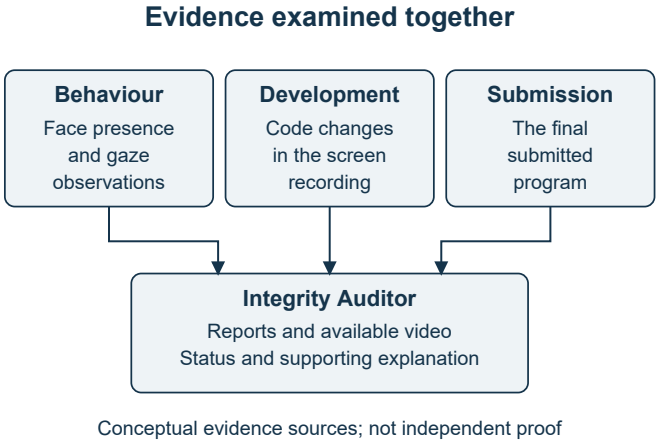


Fig. 2. Conceptual evidence sources represented in the agent reports and available recording. The submitted program is examined through the code report. Agreement between sources does not establish misconduct.

The **Code Reviewer (CR)** compares the submitted code with the screen recording and reference material. Its prompt asks for *Sudden Logic* events: complex code appearing in less than three seconds. The prompt excludes boilerplate and minor changes to test data or print statements. The three-second rule is a prompt instruction. The available evidence does not show that this threshold was optimized through experiments or that it reliably detects unauthorized assistance.

The **Integrity Auditor (IA)** receives the visual and code reports and, when available, the recording (Fig. 2). Its prompt asks for an audit status of PASS, FAIL, or INVESTIGATION, a confidence score, timestamped discrepancies, and an explanation. It asks the model to check whether sudden code appearance occurs at the same time as face absence and whether the submitted program differs substantially from the recorded program.

The auditor uses language-model prompts. Its code does not enforce a one-second matching window. A confidence score generated by the model is not a calibrated probability of misconduct. A future timing check should use timestamps aligned to the same clock and distinguish events that occur together from events elsewhere in the session. It must account for recording offsets and playback speed before applying a time tolerance. This study does not establish a validated numerical threshold for matching events.

The **Final Grader (FG)** combines the code review with the audit report. In the prototype code reviewed for this study, its prompt sets a maximum grade of 20/100 when the audit reports FAIL or INVESTIGATION. The grading rule and audit status are separate; applying the rule does not establish that an allegation is correct. Review before any integrity penalty

TABLE I
PROMPT EXCERPTS AND SELECTED REQUESTED OUTPUTS

Agent	Exact excerpt	Selected output fields
VP	"Looking at the keyboard is usually normal."	Verdict: CLEAR, SUSPICIOUS, or CRITICAL; anomaly log; reasoning.
CR	"Flag any 'Sudden Logic': complex code that appears in under 3 seconds rather than being typed incrementally."	Score; screen/submission match; sudden logic timestamps; technical analysis.
IA	"AUDIT STATUS: [PASS / FAIL / INVESTIGATION]"	Audit status; confidence score; timestamped discrepancies; reasoning.
FG	"STATUS: [Certified / Flagged / Failed]"	Final grade; status; technical score before the audit penalty; combined timeline.

is a required safeguard for deployment. These cases do not demonstrate that safeguard.

Prompt design and output fields. The prompt templates define a role, provide the available evidence, list review tasks, and request named fields in a text report. The Visual Proctor receives OpenCV logs and optional video. The Code Reviewer receives code, the rubric and retrieved references, and video when available. The auditor receives the visual and code reports plus optional video; the grader receives the audit and code reports. Table I gives short excerpts and selected output fields from the available source. These document the inspected templates; the retained case records do not confirm the exact prompt version used for each demonstration.

These are requested fields, not an enforced output schema. The shared generation call returns text, so the prompt alone does not guarantee valid labels or complete fields. The prompts also contain assumptions that need testing: the auditor instructs that frequent brief sideways gaze shifts in video described as $2\times$ speed "MUST BE FLAGGED." Such wording may encourage false alarms; its effect has not been measured. The grader requests a 20/100 cap for FAIL or INVESTIGATION and also when the audit mentions fraud. These instructions describe prototype behaviour and should not be treated as validated decision rules.

B. Retrieval Grounding and Smart Caching

The implementation reviewed for this study uses ChromaDB to retrieve reference material, with a Google embedding configuration naming `gemini-embedding-001`. This material is supplied to the Code Reviewer. The effect of retrieval on false alarms has not been measured in these cases. The summary alone does not establish the exact configuration used in the demonstration runs.

The cache stores assessment reports for reuse. A new assessment processes the submitted evidence and makes model calls. A cache hit retrieves a stored result and skips that work. Retrieving the same report repeatedly does not repeat the model analysis or show that a new run would produce the same result.

TABLE II
SELECTED CASES AND SAVED OUTCOME RECORDS

ID	Scenario	Expected label	Saved record
T1	Incremental development	PASS	Summary records PASS
T2	Sudden code appearance	FAIL	Summary records FAIL
T3	Screen/submission mismatch	FAIL	Summary records FAIL
T4	Gaze scenario, described at $2\times$ speed	FAIL	Audit excerpt: INVESTIGATION; grade capped at 20/100

IV. EXPERIMENTAL SETUP

The demonstration contains four selected case packages recorded by M. Sajid Ali, the developer and co-author, who appears in all four videos. No external students were recruited. The recordings therefore represent one person performing four scenarios. Table II summarizes the saved outcome records. The evaluation script assigns PASS to the honest scenario and FAIL to the other three scenarios. The developer assigned these expected labels. The saved summary does not document an independent labelling procedure or the number of repeated runs without cache retrieval. The unit reported here is a case package.

Project notes name Gemini 1.5 Flash and Gemini 2.0 Flash. The reviewed agent code uses a shared fallback list that starts with Gemini 1.5 Flash and tries other models if a call fails. This does not establish which model each agent used in the demonstration runs. The shared generation call does not explicitly set a temperature or seed. Prompts request labelled text reports and do not enforce a JSON output format. These details describe the reviewed source code. Records of the models and settings used in each demonstration call are still needed.

The evaluation script submits each package to the backend and searches the final report for the strings `FAIL` or `SUSPICIOUS`. If neither appears, it records PASS. This does not reliably extract the auditor's three possible statuses. An INVESTIGATION report can be assigned PASS when neither string appears. A passing report could also mention FAIL in an explanatory sentence. This extraction method therefore cannot support validated performance measurements without checking the actual audit statuses.

Code availability. The project repository is available at https://github.com/MSajidAli-byte/Exam-Flow. Release `v1.0-milestone1`, titled "IEEE Paper Initial Submission Version," is archived on Zenodo [12]. This archive provides a versioned source-code reference. It does not by itself

establish the exact settings or complete outputs of the four demonstration runs.

To reproduce the evaluation, the records must include the exact code revision, case files, prompts, model used for each call, generation settings, reference collection, raw agent outputs, and cache state. The available summary does not contain all these details. A future evaluation should read a separate status field, keep INVESTIGATION as a distinct outcome, and define how statuses map to a binary alert before scoring. It should record both failed and successful calls and distinguish repeated model analysis from cache retrieval.

V. Results and Discussion

The saved summary records PASS for T1 and FAIL for T2 and T3. For T4, the quoted audit status is INVESTIGATION, while the summary table calls the prediction FAIL and reports a grade of 20/100. Table II shows the expected label separately from the available output record. The grading cap explains how a failing grade can follow an investigation status. It does not make INVESTIGATION an exact match for the expected FAIL label.

These observations describe selected cases. They do not establish perfect classification, a detection rate, or a false-alarm rate. F1 can be calculated for a small sample when predictions and scoring rules are clear. Here, the status extraction and inconsistent outcomes prevent a reliable calculation of that score. Raw predictions and experimental settings for unimodal baselines are also unavailable. The study does not establish better performance than a visual-only or code-only baseline.

T4 also shows a timing limitation. The saved case timeline places code insertions in the first minute and face absence later in the recording. The times listed do not support a one-second match. The audit excerpt identifies sudden insertions but does not establish a validated timing link to the later absence. T4 is therefore reported as an investigation output whose timing evidence remains unresolved. The record does not establish a new rule based on the sequence of events or prove misconduct.

The three-second Sudden Logic rule remains a prompt instruction. Its sensitivity to legitimate pasting, refactoring, and different recording speeds has not been measured.

New analysis and retrieval of a stored report involve different work. Paired measurements for individual runs, repetition counts, and the points where timing started and stopped were unavailable for verification. The available records do not establish numerical latency, speedup, or cost estimates. A repeatable benchmark should measure upload, preprocessing, model analysis, deliberate waiting, and report assembly for fresh runs. It should measure cache lookup separately. The reviewed orchestrator includes three ten-second waits between agents. A stated timing method must explain whether these waits are included. Cache performance does not establish how quickly the system assesses unseen work.

VI. Discussion: Equity, Ethics, and Pedagogical Alignment

An integrity system can affect students even when its output is called a flag. The prototype's automatic grading cap makes it necessary to distinguish an observation, a suspicion, and a disciplinary decision.

A. Demographic Equity and Algorithmic Bias

The four cases were not used to compare demographic groups. Research on other visual classification tasks shows why performance across groups should be examined, but does not measure bias in Exam-Flow [10]. Combining signals may carry shared errors forward or increase suspicion. These cases do not show reduced bias or fewer false accusations. Evaluation should cover different lighting conditions, assistive technologies, working styles, and participant groups. Explanations must be checked against the source evidence; plausible wording alone is insufficient.

B. Data Privacy and Compliance

The reviewed orchestrator can upload a video file to an external model service and share the returned file reference across agents. The prototype therefore cannot be described as sending only anonymized features. This study does not verify anonymization, encrypted evidence storage, deletion guarantees, or compliance with specific laws. Before use with students, consent, access restrictions, retention limits, and deletion procedures require documented institutional review [3], [4]. Sharing a file reference avoids repeated uploads but does not guarantee privacy.

C. Pedagogical Alignment and Student Agency

An evidence report is intended to help an instructor examine a concern and seek an explanation. Whether code reuse is permitted depends on the assessment rules. Looking away alone does not establish misconduct. Students should be able to explain flagged activity and challenge an incorrect interpretation. These safeguards are proposed; their educational effects have not been measured. Before deployment, the automatic cap should be replaced with a process that preserves the technical assessment while an authorized reviewer considers the integrity allegation.

D. Limitations

Four scenarios recorded by one developer cannot establish performance across students, tasks, or institutions. The demonstration has no participant diversity. The record does not establish independent labelling, consistent results across repeated runs, or reliable handling of ordinary actions that resemble suspicious events. T1 alone is insufficient to estimate false alarms. Dedicated cases are needed for legitimate pasting, rapid refactoring, keyboard glances, face occlusion, and recording problems.

The saved summary also lacks complete records of how the runs were performed. Current source settings may differ from those used in the demonstrations. Timestamps and confidence values generated through prompts require independent checks. The effects of model fallback and caching must also be controlled. The findings support a description of the workflow and its limitations. They do not establish accuracy, fairness,

scalability, or readiness for assessment decisions that affect students.

VII. Conclusion and Future Work

Exam-Flow combines recorded behaviour, code development, and submitted programs through four agents to produce an assessment report. Four selected cases provide preliminary observations of this workflow. T4 shows why audit status, grading policy, and event timing must be reported separately. The available evidence does not establish detection accuracy or better performance than systems using one type of evidence.

The next evaluation should save complete outputs and run settings, extract statuses explicitly, and include repeated fresh analyses and legitimate activities that could trigger alerts. Timing checks should align timestamps to the same clock and document recording offsets and playback speed. Baselines should use the same cases under stated settings. Review before penalties, privacy controls, and broader evaluation are needed before the prototype can be considered for student assessment.

Acknowledgment

Large language model tools assisted with language editing and identifying inconsistencies during manuscript preparation. The authors remain responsible for the scientific claims, source records, and final submission.

References

[1] P. M. Newton and K. Essex, "How common is cheating in online exams and did it increase during the COVID-19 pandemic? A systematic review," *J. Academic Ethics*, vol. 22, pp. 323–343, 2024, doi: 10.1007/s10805-023-09485-5.

[2] A. Nigam, R. Pasricha, T. Singh, and P. Churi, "A systematic review on AI-based proctoring systems: Past, present and future," *Education and Information Technologies*, vol. 26, no. 5, pp. 6421–6445, 2021, doi: 10.1007/s10639-021-10597-x.

[3] S. Coghlan, T. Miller, and J. Paterson, "Good proctor or 'big brother'? Ethics of online exam supervision technologies," *Philosophy & Technology*, vol. 34, pp. 1581–1606, 2021, doi: 10.1007/s13347-021-00476-1.

[4] E. Zeide, "Big proctor: Online proctoring problems and how FERPA can promote student data due process," *Notre Dame J. on Emerging Technologies*, vol. 3, no. 1, pp. 74–140, 2022.

[5] L. Prechelt, G. Malpohl, and M. Philippsen, "Finding plagiarisms among a set of programs with JPlag," *J. Universal Computer Science*, vol. 8, no. 11, pp. 1016–1038, 2002.

[6] G. Lee, J. Kim, M.-s. Choi, R.-Y. Jang, and R. Lee, "Review of code similarity and plagiarism detection research studies," *Applied Sciences*, vol. 13, no. 20, Art. no. 11358, 2023, doi: 10.3390/app132011358.

[7] H. Cheers, Y. Lin, and S. P. Smith, "Academic source code plagiarism detection by measuring program behavioral similarity," *IEEE Access*, vol. 9, pp. 50391–50412, 2021, doi: 10.1109/ACCESS.2021.3069367.

[8] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *arXiv:2005.11401*, 2020.

[9] Y. Atoum, L. Chen, A. X. Liu, S. D. Hsu, and X. Liu, "Automated online exam proctoring," *IEEE Trans. Multimedia*, vol. 19, no. 7, pp. 1609–1624, 2017, doi: 10.1109/TMM.2017.2656064.

[10] J. Buolamwini and T. Gebru, "Gender shades: Intersectional accuracy disparities in commercial gender classification," in *Proc. 1st Conf. Fairness, Accountability and Transparency (FAT*)*, 2018, pp. 77–91.

[11] D. R. Garrison, T. Anderson, and W. Archer, "Critical inquiry in a text-based environment: Computer conferencing in higher education," *The Internet and Higher Education*, vol. 2, no. 2-3, pp. 87–105, 2000.

[12] M. Sajid Ali, "MSajidAli-byte/Exam-Flow: IEEE Paper Initial Submission Version," version v1.0-milestone1, Zenodo, Oct. 19, 2026, doi: 10.5281/zenodo.23465605. [Online]. Available: https://doi.org/10.5281/zenodo.23465605.